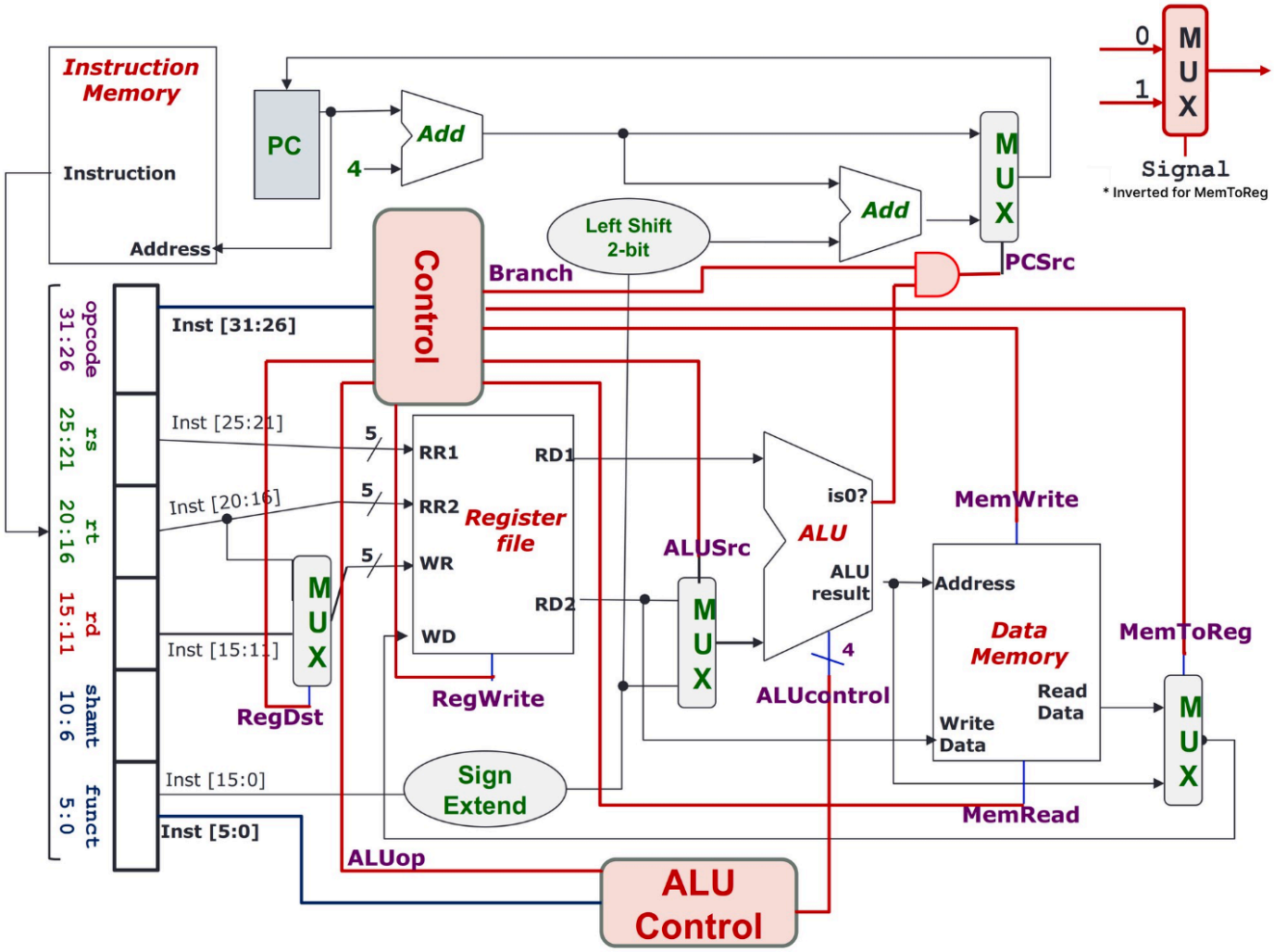


	RegDst	ALUSrc	MemTo Reg	Reg Write	Mem Read	Mem Write	Branch	ALUop	
								op1	op0
R-type	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

	Opcode (Op[5:0] == Inst[31:26])						
	Op5	Op4	Op3	Op2	Op1	Op0	Value in Hexadecimal
R-type	0	0	0	0	0	0	0
lw	1	0	0	0	1	1	23
sw	1	0	1	0	1	1	2B
beq	0	0	0	1	0	0	4

Figure 1: MIPS Control Path

MIPS Reference Data

①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr.	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		$0 / 08_{hex}$
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs] +SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs] +SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		$0 / 27_{hex}$
Or	or	R	R[rd] = R[rs] | R[rt]		$0 / 25_{hex}$
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		$0 / 2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	$0 / 2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		$0 / 00_{hex}$
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		$0 / 02_{hex}$
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (*atomic*) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	$0 / 22_{hex}$
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		$0 / 23_{hex}$

(1) May cause overflow exception
(2) SignExtImm = { 16{immediate[15]}, immediate }
(3) ZeroExtImm = { 16{1b'0}, immediate }
(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }
(6) Operands considered unsigned numbers (vs. 2's comp.)
(7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

R	opcode	rs	rt	rd	shamt	funct
	31–26	25–21	20–16	15–11	10–6	5–0
I	opcode	rs	rt	immediate		
	31–26	25–21	20–16	15–0		
J	opcode	address				
	31–26	25–0				

ARITHMETIC CORE INSTRUCTION SET

②

NAME, MNEMONIC		FOR-MAT	OPERATION		OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4)	11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr	(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]		0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6)	0/--/--/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]		11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}		11/11/--/0
FP Compare Single	c.x.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0		11/10/--/*y*
FP Compare Double	c.x.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0		11/11/--/*y*
* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)					
FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]		11/10/--/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}		11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]		11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}		11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]		11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}		11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2)	31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2)	35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi		0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo		0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]		10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]		0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6)	0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt		0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2)	39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2)	3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	fmt	ft	fs	fd	funct
	31–26	25–21	20–16	15–11	10–6	5–0
FI	opcode	fmt	ft	immediate		
	31–26	25–21	20–16	15–0		

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design,* 4th ed.

OPCODES, BASE CONVERSION, ASCII SYMBOLS

③

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexadecimal	ASCII Character	Decimal	Hexadecimal	ASCII Character
(1)	sll	add.*f*	00 0000	0	0	NUL	64	40	@
		sub.*f*	00 0001	1	1	SOH	65	41	A
j	srl	mul.*f*	00 0010	2	2	STX	66	42	B
jal	sra	div.*f*	00 0011	3	3	ETX	67	43	C
beq	sllv	sqrt.*f*	00 0100	4	4	EOT	68	44	D
bne		abs.*f*	00 0101	5	5	ENQ	69	45	E
blez	srlv	mov.*f*	00 0110	6	6	ACK	70	46	F
bgtz	srav	neg.*f*	00 0111	7	7	BEL	71	47	G
addi	jr		00 1000	8	8	BS	72	48	H
addiu	jalr		00 1001	9	9	HT	73	49	I
slti	movz		00 1010	10	a	LF	74	4a	J
sltiu	movn		00 1011	11	b	VT	75	4b	K
andi	syscall	round.w.*f*	00 1100	12	c	FF	76	4c	L
ori	break	trunc.w.*f*	00 1101	13	d	CR	77	4d	M
xori		ceil.w.*f*	00 1110	14	e	SO	78	4e	N
lui	sync	floor.w.*f*	00 1111	15	f	SI	79	4f	O
	mfhi		01 0000	16	10	DLE	80	50	P
(2)	mthi		01 0001	17	11	DC1	81	51	Q
	mflo	movz.*f*	01 0010	18	12	DC2	82	52	R
	mtlo	movn.*f*	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	]
			01 1110	30	1e	RS	94	5e	^
			01 1111	31	1f	US	95	5f	_
lb	add	cvt.s.*f*	10 0000	32	20	Space	96	60	`
lh	addu	cvt.d.*f*	10 0001	33	21	!	97	61	a
lwl	sub		10 0010	34	22	"	98	62	b
lw	subu		10 0011	35	23	#	99	63	c
lbu	and	cvt.w.*f*	10 0100	36	24	$	100	64	d
lhu	or		10 0101	37	25	%	101	65	e
lwr	xor		10 0110	38	26	&	102	66	f
	nor		10 0111	39	27	'	103	67	g
sb			10 1000	40	28	(	104	68	h
sh			10 1001	41	29	)	105	69	i
swl	slt		10 1010	42	2a	*	106	6a	j
sw	sltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
swr			10 1110	46	2e	.	110	6e	n
cache			10 1111	47	2f	/	111	6f	o
ll	tge	c.f.*f*	11 0000	48	30	0	112	70	p
lwc1	tgeu	c.un.*f*	11 0001	49	31	1	113	71	q
lwc2	tlt	c.eq.*f*	11 0010	50	32	2	114	72	r
pref	tltu	c.ueq.*f*	11 0011	51	33	3	115	73	s
	teq	c.olt.*f*	11 0100	52	34	4	116	74	t
ldc1		c.ult.*f*	11 0101	53	35	5	117	75	u
ldc2	tne	c.ole.*f*	11 0110	54	36	6	118	76	v
		c.ule.*f*	11 0111	55	37	7	119	77	w
sc		c.sf.*f*	11 1000	56	38	8	120	78	x
swc1		c.ngle.*f*	11 1001	57	39	9	121	79	y
swc2		c.seq.*f*	11 1010	58	3a	:	122	7a	z
		c.ngl.*f*	11 1011	59	3b	;	123	7b	{
		c.lt.*f*	11 1100	60	3c	<	124	7c	|
sdc1		c.nge.*f*	11 1101	61	3d	=	125	7d	}
sdc2		c.le.*f*	11 1110	62	3e	>	126	7e	~
		c.ngt.*f*	11 1111	63	3f	?	127	7f	DEL

(1) opcode(31:26) == 0

(2) opcode(31:26) == 17_{ten} (11_{hex}); if fmt(25:21)==16_{ten} (10_{hex}) *f* = s (single); if fmt(25:21)==17_{ten} (11_{hex}) *f* = d (double)

IEEE 754 FLOATING-POINT STANDARD

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$

where Single Precision Bias = 127, Double Precision Bias = 1023.

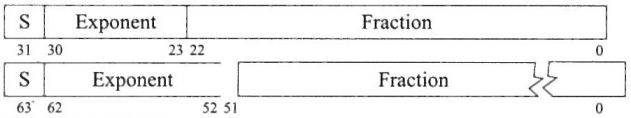
④

IEEE 754 Symbols

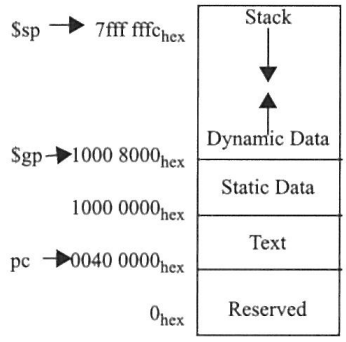
Exponent	Fraction	Object
0	0	± 0
0	≠0	± Denorm
1 to MAX - 1	anything	± Fl. Pt. Num.
MAX	0	±∞
MAX	≠0	NaN

S.P. MAX = 255, D.P. MAX = 2047

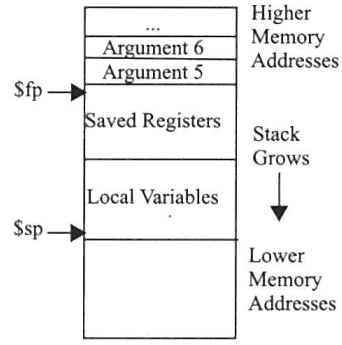
IEEE Single Precision and Double Precision Formats:



MEMORY ALLOCATION



STACK FRAME

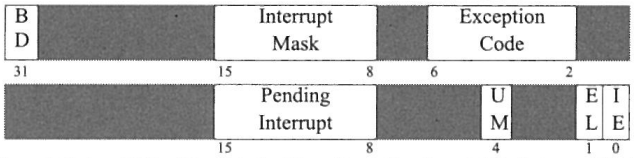


DATA ALIGNMENT

Double Word							
Word				Word			
Halfword		Halfword		Halfword		Halfword	
Byte	Byte	Byte	Byte	Byte	Byte	Byte	Byte
0	1	2	3	4	5	6	7

Value of three least significant bits of byte address (Big Endian)

EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS



BD = Branch Delay, UM = User Mode, EL = Exception Level, IE =Interrupt Enable

EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RI	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10^x for Disk, Communication; 2^x for Memory)

SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX
10^3, 2^{10}	Kilo-	10^{15}, 2^{50}	Peta-	10^{-3}	milli-	10^{-15}	femto-
10^6, 2^{20}	Mega-	10^{18}, 2^{60}	Exa-	10^{-6}	micro-	10^{-18}	atto-
10^9, 2^{30}	Giga-	10^{21}, 2^{70}	Zetta-	10^{-9}	nano-	10^{-21}	zepto-
10^{12}, 2^{40}	Tera-	10^{24}, 2^{80}	Yotta-	10^{-12}	pico-	10^{-24}	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design,* 4th ed.

Decimal - Binary - Octal - Hex – ASCII Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	“	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	‘	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	|
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

This work is licensed under the Creative Commons Attribution-ShareAlike License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/3.0/

CS2100 | Midterm Cheat Sheet

Tien Cheng & Zhao Wu

December 13, 2025

1 Appendix A: Shenanigans

1.1 C Programming

Preprocessor Directives

In C, the preprocessor processes the source code before it is compiled by compiler into assembly code.

Here are some key directives:

1. File Inclusion: `#include <filename>` (search standard directory) or `#include "filename"` (search source directory)
2. Macros: `#define MACRO_NAME SUB_VALUE`, tells preprocessor to find all instances of `MACRO_NAME` and replace with `SUB_VALUE`. Note this works recursively:

```
#define A B B B
#define B C C
#define C printf("Hello\n");

int main(void) {
  A // recursively expands to B B B => C C C C C C =>
printf("Hello\n"); x 6
}
```

3. Macros with Arguments: `#define AREA(l, b) (l * b)` works similarly to passing arguments to a function. We can use the token pasting operator `##argument_name` to join different macros:

```
#include <stdio.h>
#define paster( n ) printf_s( "token" #n " = %d",
token##n )
int token9 = 9;

int main()
{
   paster(9); // expands to printf_s( "token9 = %d",
token9 );
}
```

4. Conditional Compilation: We can use directives to include or exclude parts of the code depending on certain conditions

```
#include <stdio.h>

// Defining a macro for PI
#define PI 3.14159

int main(){

// Check if PI is defined using #ifdef
#ifdef PI
    printf("PI is defined\n");

// If PI is not defined, check if SQUARE is defined
#elif defined(SQUARE)
    printf("Square is defined\n");

// If neither PI nor SQUARE is defined, trigger an error
#else
    #error "Neither PI nor SQUARE is defined"
#endif

// Check if SQUARE is not defined using #ifndef
#ifndef SQUARE
    printf("Square is not defined");

// If SQUARE is defined, print that it is defined
#else
    printf("Square is defined");
#endif

    return 0;
}
```

Function Prototypes

- The following are valid ways to define function prototypes

```
int foo(int);
int bar(int a);

int sumArray(int []);
int sumArray(int arr[]);
int sumArray(int *);
int sumArray(int arr[4]); // size is ignored but still
valid (same goes if size is declared in function
definition)
```

- Purpose of function prototypes:
  - Let you place the function definition after any caller (C does not automatically hoist function bodies).
  - Allows compiler to verify function calls are made with the correct number and types of arguments
  - Header files (e.g `stdio.h`) generally contain function prototypes

C Operator Precedence

Precedence	Operator(s)	Associativity
1	`exp++`, `exp--`, `func()`, `arr[i]`, `.`, `->`	Left-to-Right
2	`++exp`, `--exp`, `&var`, `*ptr`, `+num`, `-num`, `!bool`, `~bits`, `(typecast)`, `sizeof`	Right-to-Left
3	`a * b`, `a / b`, `a % b`	Left-to-Right
4	`a + b`, `a - b`	Left-to-Right
5	`a << b`, `a >> b`	Left-to-Right
6	`a < b`, `a <= b`, `a > b`, `a >= b`	Left-to-Right
7	`a == b`, `a != b`	Left-to-Right
8	`a & b`	Left-to-Right
9	`a ^ b`	Left-to-Right
10	`a | b`	Left-to-Right
11	`a && b`	Left-to-Right
12	`a || b`	Left-to-Right
13	`cond ? e1 : e2`	Right-to-Left
14	`=`, `+=`, `-=`, `*=`, `/=`, `%=` , `<<=`, `>>=`, `&=`, `^=`, `|=`	Right-to-Left
15	`,`	Left-to-Right

- Parentheses `()` override precedence—use them when in doubt.
- `++i`/`--i` (prefix) yields the updated value; `i++`/`i--` (postfix) yields the old value.
- `sizeof` yields `size_t` and does not evaluate most expressions.

Mixed-Type Arithmetic

Note that in C, division between integers will perform floor division

```
int m = 10 / 4;     // m = 2 (integer division)
int n = 10 / 4.0;   // 10/4.0 is 2.5 (double); assigning
to int truncates to 2
float p = 10 / 4;   // 10/4 is 2, then promoted to 2.0f
float q = 10 / 4.0; // fully floating-point division:
2.5f
```

Type Casting

We can cast between compatible types in C

```
int ii = 5; float ff = 15.34;
float a = (float) ii / 2; // a = 2.5
float b = (float) (ii / 2); // b = 2.0, integer division
happens before the cast
int c = (int) ff / ii; // c = 3
```

Format Specifiers

The following table shows the types of placeholders available for use for the `printf` and `scanf` functions.

Placeholder	Variable Type	Function Use
`%c`	`char`	`printf`, `scanf`
`%d`	`int`	`printf`, `scanf`
`%f`	`float`, `double`	`printf`
`%f`	`float`	`scanf`
`%lf`	`double`	`scanf`
`%e`	`float`, `double`	`printf` (scientific notation)
`%p`	`type*`	`printf`
`%zu`	`size_t`	`printf`

- When printing, we can also specify the width and number of digits to display (does not work for `scanf`)
- `%5d`: integer, width of 5, right justified
- `%8.3f`: float with width of 8, 3 decimal place, right justified

Multi-Dimensional Arrays

- Memory is 1D; multidimensional arrays are linearized.
- For a multi-dimensional array `int arr[N][M][..]`, we can linearize it into a one-dimensional array using either row-major or column-major order.
- C uses row-major order: store a full row, then proceed to the next row.
- Given a multi-dimensional array of type `T`, `T a[D1][D2]...[Dn]`, where D_j is the size of dimension j, A_0 is the address of the first element `a[0][0]...[0]`, and S is the size (in bytes) of one element of `T`, the address of element $a[i_1][i_2]...[i_n]$ is

$$A_0 + S\sum_{k=1}^{n}\left(i_k \prod_{m=k+1}^{n} D_m\right)$$

(by convention the empty product equals 1).

Unions

- A `union` reuses the same memory for all its members; its size is the max of its members' sizes;
- Purpose: To save memory by allowing different data types to use the same storage space at different times (like a generic variable).
- Only one member can hold a value at any given time. Assigning a value to one member overwrites the values of all other members.

```
typedef union {
  int    i;
  float  f;
  void  *p;
} Cell;

Cell c = { .i = 42 }; // initialize the 'i' member
c.f = 3.14f;          // now the active interpretation
is 'f'
```

Enums

- An `enum` is a C type that represents a group of (integer) constants

```
enum EnumName {
    CONSTANT1 = 1, // if value not specified, default to
0
    CONSTANT2, // auto increments to 1 + 1 = 2
    CONSTANT3 // auto increments to 2 + 1 = 3
};
```

Static Global Variables and Function

- A static global function is only visible and callable within the source file where it is defined. It cannot be called by functions in other source files.
- A static global variable is only visible and accessible within the source file where it is defined. It cannot be accessed by functions or variables in other source files, even if those files are linked together in the same program.

C Quirks

- In C, `%` is the remainder operator, so `-10 % 4` yields `-2` (it keeps the sign of the dividend).
- Any nonzero integer or non-null pointer is treated as `true`; `_Bool`/`bool` values print as `0` or `1`.
- Strings cannot be used as a case for a switch statement
- The following are equivalent: `arr[3]` and `3[arr]` due to the commutativity of the de-referencing `*` operator
- Arrays initialized with string literals (e.g., `char str[] = "Hello";`) are modifiable, while pointers to string literals (e.g.,

`char *str = "Hello";`) point to read-only memory and should not be modified.
- Prefixes: `0` for octal, `0x` for hex; `0b` for binary is a GCC/Clang extension
- C is statically typed but not strongly typed—implicit conversions happen frequently, so the compiler will not catch every type-mismatch bug for you.
- Declaring a pointer does not allocate the storage for the pointee; even `char *p = "hi";` merely points at existing (read-only) storage
- Global variables exist for the full program and are unique application-wide (linker detects duplicates across files).
- You can have a pointer to a pointer

Unexpectedly Testable C Facts

- **Partial aggregate initialization**: you can list fewer initializers than members; the remaining fields zero-initialize.

```
struct Triple { int x, y, z; };
struct Triple t = { .y = 7 };  // x and z become 0 automatically
int arr[5] = { 1, 2 };         // remaining slots set to 0
```

- **Designated initializers can appear out of order** as long as each field is named once; handy for sparse structs.
- **Compound literals have block scope lifetime** (static if at file scope, automatic if inside a function). Borrowing their address after the full expression ends is UB.

```
int *p = (int[]){ 1, 2, 3 }; // array storage disappears right after the statement
```

- **`sizeof` never evaluates its operand** (unless the operand has VLA type), so `sizeof(i++)` does not increment `i`. The result type is `size_t`.
- **`char` signedness is implementation-defined**. Guard code that depends on negative `char` values with explicit `signed char`.
- **Order of evaluation of function arguments and most subexpressions is unspecified**. Expressions like `f(i++, i++)` are undefined behavior because `i` is modified twice without a sequence point.
- **Pointer dereference vs. increment precedence**: postfix `++` binds tighter than unary `*`, so `*p++` means `*(p++)` (use value, then advance pointer), while `*++p` means `*(++p)` (advance first, then read). Parenthesize `(*p)++` if you intend to increment the pointee instead of the pointer. The same rules apply to `--`.
- **Flexible array members** (`struct Foo { size_t len; int data[]; };`) must be the last member, cannot have an initializer, and require over-allocation via `malloc(sizeof(Foo) + len * sizeof(int))`.
- **Static locals retain state across calls** and zero-initialize once: useful but also a test favorite.

1.2 Number Systems and Data Representation

IEEE 754 Reserved Values

The IEEE 754 standard reserves certain exponent values to represent special numbers

Exponent	Mantissa	Represents
255	0	Infinity ($\pm\infty$)
255	$\neq 0$	Not a Number (NaN)
0	0	Zero (± 0)
0	$\neq 0$	Denormalized Number

- Denormalized is used for very small values with fixed exponent

IEEE 754 Rounding

- During operations, the hardware keeps 3 extra bits beyond the stored fraction:
  - G (guard), R (round), S (sticky), these are the 3 bits immediately to the right of the mantissa bits
  - Rounding is used when the number cannot be fully represented in IEEE 754, so we need to round up or down to the nearest representable number

```
if (GUARD == 1) {
  if (ROUND == 1 || STICKY == 1) {
    // Round Up
    return ROUND_UP(X);
  }

  // Invoke LSB (bit 23) as tie-breaker
  if (LSB == 1) {
    return ROUND_UP(X);
  }
}
// Default is
return ROUND_DOWN(X);
```

- Implication: floating point numbers are not associative in practice

Max/Min Without a Hidden Bit

Consider a floating-point format with 1 sign bit, 6 exponent bits, and 9 stored mantissa bits (no hidden bit). The exponent bias is

$$B = 2^5 - 1 = 31$$

Assuming every 6-bit exponent pattern is used for normalized numbers, the largest and smallest exponents are

$$E_{\max} = (2^6 - 1) - B = 32 \text{ and } E_{\min} = 0 - B = -31$$

The largest positive finite value sets every mantissa bit to 1:

$$\max = 0.111111111_2 \times 2^{32} = 2^{32} - 2^{23} = 111111111_2 \times 2^{23} = 4286578688$$

The smallest positive normalized value uses the smallest mantissa and exponent:

$$\min_{\{>0\}} = 0.100000000_2 \times 2^{\{-31\}} = 2^{\{-32\}}$$

The most negative finite value is $-\max$.

IEEE 754 **does** use a hidden bit, so its mantissa is interpreted as $\text{sign}(1.\text{fraction})_2 \times 2^{\text{exponent}}$. In this custom format without the hidden bit, we instead evaluate mantissa bits directly as $\text{sign}(\text{mantissa})_2 \times 2^{\text{exponent}}$.

Binary Coded Decimal

- Encodes each decimal digit separately using a 4-bit binary pattern (`0000`..`1001`). Patterns `1010`–`1111` are invalid and often treated as error indicators.
- Useful when decimal digit boundaries must be preserved (e.g., financial data) because scaling by powers of ten only requires shifting by nibbles, not reinterpreting the binary magnitude.

84-2-1 Code

- A weighted self-complementing code whose bit positions carry weights $8, 4, -2, -1$. Valid combinations still cover decimal digits 0–9, but the negative weights force unused bit patterns to show up as invalid digits.
- Because the code is self-complementing, the 9's complement of a digit can be obtained by flipping every bit, which simplifies subtraction hardware.

2421 Code

- Another self-complementing weighted code with weights $2, 4, 2, 1$. Most digits have two different legal encodings; designers usually pick the canonical one that yields even parity.
- As with 84-2-1, taking the bitwise complement immediately yields the 9's complement of the digit.

Biquinary Code

- Represents each decimal digit with two fields: a 2-of-5 field that marks which group (0–4 or 5–9) we are in, and a 1-of-5 field that encodes the position within that group.
- The redundancy (two bits set in every code word) allows single-bit error detection—any pattern with the wrong number of ones is invalid.

Parity Bit

- Append one bit so the total number of ones is even (even parity) or odd (odd parity). The receiver recomputes the parity and flags an error if it disagrees.
- Detects any single-bit error but cannot correct it and may miss multi-bit flips that preserve parity.

Hamming Code

- Adds multiple parity bits whose positions are powers of two (1, 2, 4, 8, ...). Each parity bit covers a unique subset of data bits.
- When a code word is read back, the parity bits form a binary syndrome that identifies the bit position that flipped, allowing single-bit error correction and double-bit error detection.

1.3 MIPS

Facts

- MIPS registers are untyped—any 32-bit pattern can be interpreted as an integer, pointer, or float depending on the instruction that reads it.
- MIPS: Microprocessor without Interlocked Pipelined Stages
- MIPS has 32 general-purpose registers, each 32 bits wide.
- Memory is byte-addressed with 32-bit addresses: 2^{32} bytes total = 2^{30} words (a word is 4 bytes).
- Endianness is implementation-specific; classic MIPS systems were big-endian, but many embedded cores are little-endian. Know which mode your simulator uses.
- `$at` register: usually used by assembler to store values for pseudo-instructions, hence not recommended to use in your own code
- Labels don't count as instructions. When used in MIPS code, the assembler internally converts labels to either the actual memory address (for jump instructions) or as offset (for branch instructions)

Instructions

- `srl` vs `sra`: `srl` will fill the new MSB with 0, while `sra` does sign extension, so it will fill with 1 for negative numbers
- `andi`, `xori` and `ori` performs zero-extension of the 16 bit immediate. Note that if you perform `andi` against a value that has bits in the upper 16 bits, then it will override those bits with 0
- `addi` and `slti` performs sign extension of the 16-bit immediate
- `lb` and `sb` are commonly used when processing strings, since `chars` are single byte in size. When processing strings, we should take care not to increment our iterator by 4. If we use `lw` to process strings, we will load every 4 characters instead.
- lw/sw require word-aligned addresses (address % 4 == 0). Unaligned lw/sw cause exceptions.
- Pseudo-instructions for unaligned word access expand into sequences of real instructions (e.g., using lb/sb); avoid them when possible.
  - `ulw` and `usw` (unaligned load word and unaligned store word)
- Other memory instructions:
  - `lh` and `sh`: load halfword and store halfword
  - `lwl`, `lwr`, `swl`, `swr`: load/store word left/right
- `addi` traps on signed overflow while `addiu` does not. Signed compare instructions (`slt`, `slti`) treat operands as signed; the `u` variants (`sltu`, `sltiu`) perform unsigned comparisons even though immediates are still sign-extended.
- `jal` writes `PC + 8` (the address of the instruction after the delay slot) into `$ra`; `jr $ra` therefore returns to the instruction **after** the delay slot.

1.3.1 Common MIPS Patterns

Loading 32 Bit Constants

1. Use `lui` to set the upper 16 bits (e.g `lui $t0, 0xAAAA`)

- `lui` will fill the lower bits with 0s as it is a pseudo-instruction that first loads then shifts the value left

2. Use `ori` to set the lower-order bits (e.g `ori $t0, $t0, 0xF0F0`)

Implementation of NOT operation

- We can implement NOT operation using either `NOR` or `XOR` operations
  - `nor $t0, $t0, $zero`
  - `xor $t0, $t0, $t1` assuming `$t1` is an all ones mask (requires loading a 32 bit constant with all ones e.g `addi $t1, $zero, -1`)

Modulo of Power of 2

- Suppose we want to calculate the modulo of a power of 2, $x \bmod 2^n$, we can use the formula $x \bmod 2^n = x \wedge (2^n - 1)$

```
# Assume $s0 holds the number (e.g., 45)
li   $t1, 15        # Create the bitmask (16-1=15, or
```

```
0b00001111)
and  $t0, $s0, $t1  # $t0 = $s0 AND 15 (45 & 15 = 13)
```

Checking for Word Alignment

- An address is word aligned if the value of the address modulo the word size is 0
- So, if we assume that our word size is a power of 2, we can take the bitwise and with the word size - 1 and check if the result is 0
- Recall a word is 4 bytes, so we can just do `andi $t1, $a0, 3 # 4 - 1 = 3`

Indexing Based Iteration of Arrays

- Idea: use an index variable, and calculate the exact memory address (base address + offset) inside the loop during each iteration

1. Store the index variable i and base address in a register, set $i = 0$
2. Within each iteration, calculate the offset $i \times 4$: `sll $t1, $t0, 2$`
3. Calculate the address of the current element by adding the base address: `add $t2, $a0, $t1`
4. Access element $A[i]$: `lw $s3, 0($t2)`
5. Increment the index: `addi $t0, $t0, 1`

Pointer Based Iteration of Arrays

- Idea: Avoid repeated multiplication and addition inside the loop by maintaining a register that holds the current memory address, which is incremented directly by the size of the data type
- This results in more efficient code due to less instructions per iteration

1. Initialize the pointer p: `addi $t0, $a0, 0`
2. Calculate the end address:
   - `sll $t1, $a1, 2` (size x 4)
   - `add $t2, $a0, $t1` (t2 stores end address)
3. Within the loop body:
   - Access element $A[i]$ (dereferencing): `lw $s3, 0($t0)`
   - Increment pointer p: `addi $t0, $t0, 4`
   - Check loop condition (blt):
     - `slt $t3, $t0, $t2`
     - `bne $t3, $zero, Loop`

Jumping Across Jump Range

- We can use branch instructions to allow for jumps across the 256MB memory region

```
      beq  $t0, $t1, MidBranch      # If $t0 == $t1, go
to MidBranch.

      # Code that runs if the branch is NOT taken ($t0 !
= $t1).
      addi $s0, $s0, 1              # Increment a
counter.
      sub  $s1, $s1, $s0            # Perform some other
work.

      # Prevent accidental execution of bridge
      # This is an unconditional jump. It's always true
($zero == $zero).
      # It acts as a "fence" to hop over the special
MidBranch code.
      beq  $zero, $zero, AfterFix   # Always jump to the
'AfterFix' label.

# --- Fenced-Off Area ---
# This code can now ONLY be reached if the first beq
instruction
# explicitly jumps to it. It will never be executed by
"falling through".
MidBranch:
      j    FarLabel                 # The "connecting
flight".
# -----------------------

AfterFix:
      # Normal program execution continues here.
      # ... more code ...

# ... lots of other instructions ...

FarLabel:
      # ... code at the final destination ...
```

1.3.2 Conversion Strategies

MIPS-to-C Translation (Loop Blocks)

- When given MIPS and asked for C:
  1. Identify the **prologue** that sets up base addresses, loop bounds, and accumulators; these become variable declarations and initializations.
  2. Locate the main loop: the label, the conditional branch that exits, and the jump back; reconstruct a `for`/`while` with the same condition and update.
  3. Recognise address patterns: sequences like `sll $t1,$t0,2` then `add $t2,$s0,$t1` then `lw $t3,0($t2)` correspond to `A[i]`.
  4. Rewrite the body as clean C using array notation and structured control flow, not a literal line-by-line translation.

C-to-MIPS Translation (Loop Template)

- When given C and asked for MIPS:
  1. Map long-lived variables (loop bounds, pointers, counters) to `$s` registers; use `$t` registers for temporaries.
  2. For `for (i = 0; i < n; i++)` loops, use a standard pattern:
     - Initialize `i` (`add $t0, $zero, $zero`), compare with `n` via `slt`, and use `beq`/`bne` to exit.
  3. Implement `A[i]` / `B[i]` with the scale–index–add pattern: `sll` (by 2 for words) ⟶ add base ⟶ `lw`/`sw`.
  4. Keep the loop structure obvious: test at the top, fall through into the body, then perform the increment and jump back to the loop label.

Use of Temporary Registers

- C expressions involving more than two operands must be broken down into multiple smaller statements

```
# a = b + c - d;
add $t0, $s1, $s2
sub $s0, $t0, $s3
```

Inversion of Branch Conditions

- To make the MIPS code shorter, we often invert the condition in a branch condition
- For an `if` statement:

```
if (i == j) {
  // then
}
// else
```

```
bne $s3, $s4, Else
# then
Else:
```

- For a `while` loop:

```
while (j == k) {
}
```

```
Loop:
  bne $s4, $s5, Exit
  # then
  j Loop
Exit:
```

Handling Inequalities

- Inequalities are handled using the Set on Less Than `slt` instruction
- The `slt $rd, $rs, $rt` sets register `$rd` to 1 if `$rs < $rt` and 0 otherwise
- We can then use this to implement pseudo-instructions like `blt`, `ble`, `bgt`, `bge`

C Code	Pseudo-instruction	MIPS Instruction
`if (i < j)`	`blt $rs, $rt, L`	`slt $t0, $rs, $rt` `bne $t0, $zero, L`
`if (i >= j)`	`bge $rs, $rt, L`	`slt $t0, $rs, $rt` `beq $t0, $zero, L`
`if (i <= j)`	`ble $rs, $rt, L`	`slt $t0, $rt, $rs` `beq $t0, $zero, L`
`if (i > j)`	`bgt $rs, $rt, L`	`slt $t0, $rt, $rs` `bne $t0, $zero, L`
`if (i == j)`	`beq $rs, $rt, L`	`beq $rs, $rt, L`
`if (i != j)`	`bne $rs, $rt, L`	`bne $rs, $rt, L`

MIPS Swap Procedure

```
# code to swap k with k + 1
swap:
  sll $2, $5, 2 //  k * 4
  add $2, $4, $2 // add base addr to get arr[k]
  lw $15, 0($2) // load arr[k]
  lw $16, 4($2) // load arr[k + 1]
  sw $16, 0($2)
  sw $15, 4($2)
```

1.4 Tracing Strategies

Counting Number of Instructions with Loops

- Suppose A is the number of instructions before the loop, B is the number of instructions and n is the number of iterations
- Then the number of instructions executed will be $A + \text{Bn}$
- Note that for conditional branches with `BNE`, the branch instruction executes n times, but branches are taken $n - 1$ times

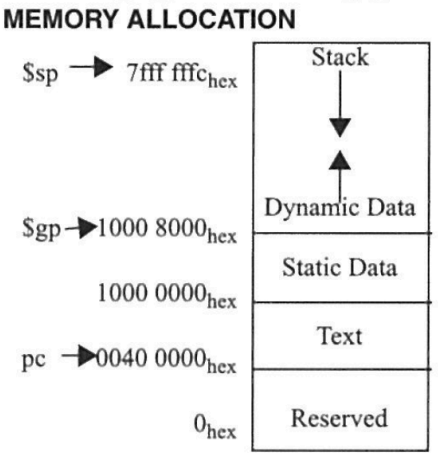
Counting Number of Unique Bytes Read

```
       addi $t1, $t0, 10
       add  $t2, $zero, $zero
Loop:  ulw  $t3, 0($t1) # ulw: unaligned lw
       add  $t2, $t2, $t3
       addi $t1, $t1, -1
       bne  $t1, $t0, Loop
```

- `ulw` loads 4 bytes at a time, but the pointer is incremented by 1 each iteration, meaning each load overlaps with the previous one, except for the first.
- The first `ulw` accesses 4 unique bytes; each subsequent iteration only reads one additional unique byte.
- So if there are `n` iterations, we read only $4 + (n - 1)$ unique bytes.

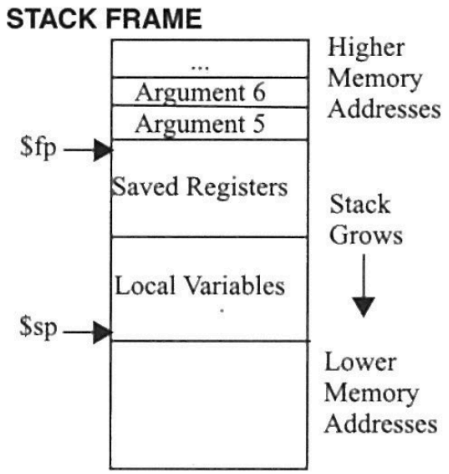
1.5 Procedure Calls

Stack Operation in MIPS



- In MIPS, the stack grows from a high memory address towards a lower memory address
- The **stack pointer register** `$sp` always points to the top-most valid data item on the stack (the lowest memory address currently in use by the stack)
- Push Operation (Allocating Space): to allocate space, decrement the stack pointer, then perform a store operation with the stack pointer as the base address
- Pop Operation (Deallocating Space): to deallocate space, increment the stack pointer by the number of bytes you want to deallocate (e.g `addi $sp, $sp, 8` frees 8 bytes)

Stack Frame in MIPS



- When a function is called, it creates a **stack frame** on the stack. This is a block of memory used for its arguments, saved registers, and local variables.
- Frame Pointer (`$fp`): points to the start of the frame, acting as a stable reference point for accessing local variables and arguments, even as the stack pointer moves to accommodate further nested calls
- Return Address (`$ra`): contains the address in the caller's code to which the callee should return. In the event that a function recursively calls other functions, the callee function should save a copy of the original return address as recursive calls will override the return address (assuming using `jal` instruction)
- Saved Registers: Locations to store the original values of any callee-saved (`$s0-$s7`) registers that the callee intends to modify

jal and jr Instructions

- `jal label` (jump and link) does the following:
  - Saves the address of the instruction immediately following the `jal` (i.e PC + 4) into `$ra`
  - Jumps to the instruction located at `label`
- `jr $rs` (jump register): Jumps to the address specified in the specified register `$rs`. Usually this is used at the end of a function to `jr $ra`, to transfer control of the function back to the caller, completing the procedure call

Procedure Calls in MIPS

```
myfunc:
    # Prologue: Setup the stack frame
    addiu $sp, $sp, -<frame_size>   # Allocate space for
the frame
    sw $ra, <offset>($sp)          # Save return address
if not a leaf procedure
    sw $s0, <offset>($sp)          # Save any callee-
saved registers ($s0-$s7)
    # ...

    # Procedure Body
    # ...
    jal some_other_proc            # Call another
procedure
    # ...

    # Epilogue: Tear down the stack frame and return
    lw $s0, <offset>($sp)          # Restore callee-
saved registers
    lw $ra, <offset>($sp)          # Restore return
address
    addiu $sp, $sp, <frame_size>   # Deallocate the
frame
    jr $ra                         # Return to caller
```

1.6 ISA

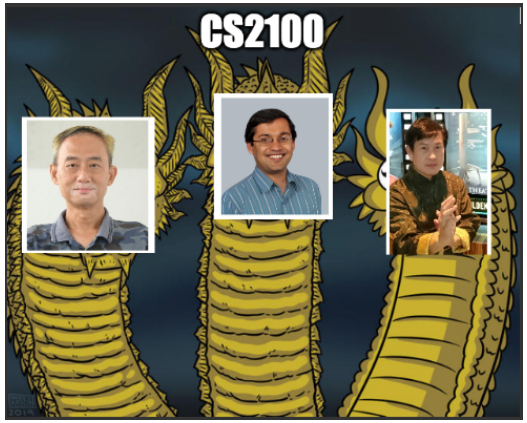
- Programmer writes program in high-level language like C -> Compiler translates to assembly -> Assembler translates to machine code -> Processor executes machine code
- Stored-program (von Neumann) model: instructions and data are both just bits stored in memory; CPU fetches/executes in program order logically, even though real CPUs may execute out-of-order internally.

Instruction Format Gotchas

- R-format opcodes are all zero in MIPS, so decoding relies entirely on `funct`; control logic must still look at both fields in case a future ISA revision reuses opcodes.
- Logical immediates (`andi`, `ori`, `xori`) zero-extend while arithmetic immediates (`addi`, `slti`) sign-extend. Mixing them up silently corrupts the upper 16 bits.
- Jump-format targets keep the upper 4 bits of `PC + 4`; if your destination crosses a 256 MB boundary you must stitch together two jumps (e.g., `lui` + `jr`) or the hardware will land in the wrong segment.

ISA Metric Curveballs

- CPI questions often hide different instruction mixes: `CPI = Σ (frequency × cycles per instruction)`. A small number of slow instructions can dominate if their latency is high, so recompute CPI whenever the mix changes.
- Structural hazard questions love the single-memory assumption: a unified memory shared between instruction fetch and data access automatically produces a structural hazard unless you add a Harvard split or insert stalls.

1.7 Misc Fun Facts

- Lectures held at: UT-AUD2
- TOTO Example (Recitation 1)
- Stage floor not even (Recitation 6)
- Pentium 4 has 22 pipeline stages